

Suitably Impressive Thesis Title



Candidate Number: 1234567

University of Oxford

Word Count: 15000

A thesis submitted for the degree of

Master of Science in Advanced Computer Science

Trinity 2026

Abstract

Your abstract goes here. Check your departmental regulations, but this is usually expected to be under 300 words — roughly one page.

A workable structure is one paragraph per move: (1) context and why the problem matters, (2) the gap, (3) what you did, (4) what you found, and (5) what it means more broadly.

Acknowledgements

Thank your supervisors first, then anyone whose ideas or feedback shaped the direction of the work.

Then colleagues, collaborators, and friends.

Then any funding or institutional support you received.

Finally, family.

Contents

1	Introduction	1
1.1	Problem Setting and Research Gap	2
1.2	Proposed Approach and Main Findings	3
1.3	Contributions	4
1.4	Thesis Overview	5
2	Background	6
2.1	Generative Modelling	6
2.1.1	Conditional Generative Modelling	7
2.1.2	Inverse Problems	7
2.1.3	Relation to Reward Alignment	8
2.2	Flow-Based Generative Modelling	9
2.2.1	Normalising Flows	9
	Bibliography	10
A	Supplementary Material	13
A.1	Change of Variables	13
A.2	Score functions and score matching	13
A.3	Additional Experiments	13
A.3.1	GMM Experiments	13

List of Figures

Chapter 1

Introduction

One goal of generative modelling is to learn how to produce novel samples that resemble those drawn from an unknown data distribution p_{data} . Diffusion and flow models [1–5] have been highly successful at this task, achieving state-of-the-art results in image, video and audio generation [6–8]. Both use the same high-level framework for generation: samples $\mathbf{z} \sim p_{\text{base}}$ drawn from a simple noise distribution are transformed continuously over time towards the data distribution [9].

Flow and diffusion models specify the dynamics of this continuous transformation between noise and data as governed by an ordinary differential equation (ODE) and a stochastic differential equation (SDE), respectively [4, 10]. An ODE defines a path from a noise to data sample by specifying the local velocity at each point along it. In a flow model, a neural network is trained to predict this velocity. Generating data therefore consists of iteratively advancing a sample from noise to data through a sequence of small steps, querying the neural network at each step to determine the direction of travel. Consequently, although flow models enable high-fidelity generation, they can often require hundreds of neural network evaluations [3], which can be computationally prohibitive.

Flow maps [11, 12] address this limitation by training a neural network to predict the cumulative displacement between any two times along the noise-to-data paths defined by the ODE, rather than the local velocity at a single point. A flow map can therefore transport a noise sample directly to a data sample in a single neural network evaluation. Flow maps can be learned by bootstrapping from a pre-trained flow model via distillation [11, 13]. However, it is also possible to train a flow map from scratch, as demonstrated by MeanFlow [14].

1.1 Problem Setting and Research Gap

Many applications require conditional generation: producing samples from the data distribution that satisfy an external condition, such as a class label or text prompt [6, 15, 16]. A canonical example of conditional generation is inverse problem solving [17], in which an observation $\mathbf{y}$ is generated by corrupting an unknown data sample $\mathbf{x} \sim p_{\text{data}}$ according to $\mathbf{y} = \mathcal{A}(\mathbf{x}) + \boldsymbol{\eta}$, where $\mathcal{A}$ is a known forward operator and $\boldsymbol{\eta}$ represents measurement noise. In image inverse problems, for example, $\mathcal{A}$ may downsample, blur or mask an image. Since these operations typically discard information, multiple data samples may be consistent with the same observation. The goal of inverse problem solving is therefore to draw samples from the posterior $p_{\text{data}}(\mathbf{x} \mid \mathbf{y})$, which represents the distribution of plausible data samples consistent with the given observation $\mathbf{y}$.

One approach to conditional generation is to train a generative model specifically for the conditional task, either from scratch or by fine-tuning a pretrained unconditional model [16, 18]. This can provide fast conditional generation, including one-step generation with a flow map [14, 19]. However, adapting the generative model to each new conditional generation setting typically requires a new round of training or fine-tuning, which can be prohibitively expensive for large generative models.

Guidance-based methods offer an alternative for flow (and diffusion) models. These methods keep the pretrained unconditional flow model frozen but nudge its velocity prediction at each sampling step, thereby steering the path from noise towards a data sample that satisfies the given condition [20–23]. This approach hence allows for same unconditional model to be reused across different conditional generation settings without fine-tuning.

However, this guidance strategy does not transfer naturally to one-step generation with a flow map. A flow map transports an initial noise sample directly to a data sample in a single evaluation, leaving no intermediate predictions through which guidance can be applied. The key insight of Variational Flow Maps (VFM) [24] is that generation can instead be controlled through the choice of starting noise. For a one-step unconditional flow map, the starting noise is the only remaining lever for controlling its output. Conditional generation can therefore be reframed as learning a distribution over starting noise samples that the flow map sends to data samples satisfying the given condition.

Let $p_{\theta}(\mathbf{x})$ denote the model distribution (which approximates $p_{\text{data}}(\mathbf{x})$) obtained by drawing $\mathbf{z} \sim p_{\text{base}}$ and passing it through the flow map, where θ denotes

the neural-network parameters of the flow map. For a given condition $\mathbf{y}$, VFM formulates the required noise distribution as the noise-space posterior $p_\theta(\mathbf{z} \mid \mathbf{y})$. Intuitively, this distribution is obtained by pulling the corresponding model posterior $p_\theta(\mathbf{x} \mid \mathbf{y})$ backwards through the flow map into noise space.

Although $p_\theta(\mathbf{z} \mid \mathbf{y})$ cannot be sampled from directly, its density can be evaluated up to a normalising constant, making variational inference (VI) [25] a natural approach for approximating it. VI introduces a tractable distribution $q_\phi(\mathbf{z} \mid \mathbf{y})$, called a variational distribution, with parameters ϕ , to approximate this posterior. Fitting $q_\phi(\mathbf{z} \mid \mathbf{y})$ using the variational objective requires both being able to sample from it and evaluate its density. VFM satisfies these requirements by parameterising $q_\phi(\mathbf{z} \mid \mathbf{y})$ as a diagonal Gaussian whose mean and variance are predicted from $\mathbf{y}$ by a neural network, termed the noise adapter.

Although a Gaussian supports both sampling and exact density evaluation, it may be too simple to approximate the potentially complex noise-space posterior $p_\theta(\mathbf{z} \mid \mathbf{y})$. VFM compensates for this by jointly training the noise adapter and the flow map, allowing the geometry of the noise space to be reshaped such that $p_\theta(\mathbf{z} \mid \mathbf{y})$ can be more easily approximated by a simple Gaussian distribution. VFM therefore enables one-step conditional generation, but only by fine-tuning the flow map. This leaves open whether one-step conditional generation can be achieved while keeping the unconditional flow map fixed.

1.2 Proposed Approach and Main Findings

The central question of this project is whether a more expressive variational distribution $q_\phi(\mathbf{z} \mid \mathbf{y})$ can approximate $p_\theta(\mathbf{z} \mid \mathbf{y})$ while leaving the pretrained flow map frozen. We investigate this question by replacing VFM’s Gaussian noise adapter with a conditional normalising flow [26, 27]. Normalising flows support both sampling and exact density evaluation, satisfying the requirements of variational training while being capable of representing substantially more complex distributions than a Gaussian. When amortising over observations $\mathbf{y}$, the inverse-problem setting also naturally permits the approximating distribution, $q_\phi(\mathbf{z} \mid \mathbf{y})$, to be learned via maximum likelihood estimation. We therefore introduce a training objective that combines the variational loss with a maximum-likelihood loss [28], and show that the combination is more effective than either loss alone.

The resulting approach enables one-step conditional generation using an

existing pretrained unconditional flow map while keeping its parameters fixed. Solving each new conditional generation setting using this approach requires training only a lightweight noise adapter. We evaluate the method across inverse problems of increasing difficulty, progressing from a two-dimensional checkerboard problem to image brightness, $4\times$ super-resolution and inpainting. On the two-dimensional problem, the Gaussian adapter fails to approximate the noise-space posterior accurately, whereas the normalising-flow adapter captures its multimodal structure. For $4\times$ super-resolution, our method achieves the lowest maximum mean discrepancy (MMD) among all evaluated baselines, including VFM.

However, we find that the approach is not competitive on image inpainting. Our analysis suggests that the main difficulty is learning, in a single step, which initial noise samples will map to images matching an observation that constrains many pixel values. This motivates training the adapter to predict a point partway along the noise-to-data path, rather than the initial noise itself, before mapping that point to data in a single flow-map evaluation. We leave the development of this extension to future work.

1.3 Contributions

This thesis makes the following contributions:

1. We adapt the Variational Flow Maps (VFM) framework for one-step conditional generation so that it is training-free with respect to the flow map. In particular we replace the Gaussian noise adapter with a more expressive normalising flow. To our knowledge, this is the first work to investigate solving conditional generation entirely within the noise space of a pretrained unconditional flow map without fine-tuning the flow map itself.
2. We further extend VFM by introducing a training objective that combines both the variational loss with a maximum-likelihood loss. We show that this combined objective is more effective than training the noise adapter with either maximum likelihood or the variational loss alone.
3. We introduce a principled evaluation framework that isolates error attributable to the trained noise adapter from error attributable to the frozen pretrained flow map.

4. We demonstrate that the more expressive normalising flow adapter can substitute for flow map fine-tuning on two-dimensional checkerboard, image-brightness and $4\times$ super-resolution inverse problems. On super-resolution, our method achieves the lowest MMD among all evaluated baselines, including VFM. However, we find that it is not competitive for image inpainting, and we provide a detailed analysis of its failure.

1.4 Thesis Overview

Chapter 2

Background

This chapter introduces the background required to understand the proposed method. We first formalise the goals of generative modelling and conditional generative modelling. We then introduce inverse problems as the conditional generation setting considered in this thesis and relate them to the broader framework of reward alignment. Next, we review the flow-based generative models on which the project builds upon: normalising flows, flow models trained through flow matching, and flow maps. Finally, we introduce variational inference and Variational Flow Maps (VFM), which our approach directly builds on.

2.1 Generative Modelling

Suppose we wish to generate examples of a particular class of object. This class may be defined as broadly or narrowly as we like. We restrict our attention to objects that can be represented as vectors $\mathbf{x} \in \mathbb{R}^D$. In generative modelling, we assume that there exists a target probability distribution $p_{\text{target}}(\mathbf{x})$ which places high density on vectors corresponding to plausible examples of the specified class of object. The goal of generative modelling is to learn an algorithm that can produce samples from this target distribution.

Often, the target distribution is a data distribution $p_{\text{data}}(\mathbf{x})$ over, for example, images, videos or molecular structures. In this setting, we typically have access to a finite dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ of samples from the data distribution. Using this dataset, the aim is to learn a model distribution $p_{\theta}(\mathbf{x})$ that approximates $p_{\text{data}}(\mathbf{x})$. Depending on the generative modelling method, it may support sampling from $p_{\theta}(\mathbf{x})$, evaluating its density, or both.

Many generative models, such as generative adversarial networks (GANs)

[29] and the flow-based generative models [4, 14, 27] considered in this thesis, generate data by first drawing noise $\mathbf{z} \sim p_{\text{base}}$ from a simple base distribution and then transforming it into a data sample $\mathbf{x}$. The learned transformation should induce a model distribution $p_{\theta}(\mathbf{x})$ that approximates the target distribution.

2.1.1 Conditional Generative Modelling

Often, we do not wish to generate an arbitrary sample from the target distribution. Instead, we wish to generate samples that satisfy a given condition $\mathbf{y} \in \mathbb{R}^M$, where $\mathbf{y} \sim p_Y(\mathbf{y})$. For example, p_Y may be a distribution over class labels, or text prompts. This task is known as conditional generative modelling. For a given condition $\mathbf{y}$, the task can be formalised as sampling from the conditional target distribution $p_{\text{target}}(\mathbf{x} \mid \mathbf{y})$ [10].

Typically, we do not want an algorithm that can generate samples for only one fixed condition. Instead, the aim is to learn an algorithm that can generate samples for different conditions $\mathbf{y} \sim p_Y(\mathbf{y})$. For example, an image generator may be required to accept different text prompts.

2.1.2 Inverse Problems

Inverse problem solving is a canonical example of conditional generation and is the focus of this thesis. To define an inverse problem, we first define the corresponding forward problem. Let $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ be a data sample and let $\mathcal{A} : \mathbb{R}^D \rightarrow \mathbb{R}^M$ be a known forward operator. The forward operator acts on $\mathbf{x}$ to produce an observation

$$\mathbf{y} = \mathcal{A}(\mathbf{x}) + \boldsymbol{\eta}, \quad \boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \sigma_y^2 \mathbf{I}_M) \quad (2.1)$$

where $\boldsymbol{\eta}$ represents measurement noise. In image inverse problems, for example, $\mathcal{A}$ may blur, downsample or mask an image. The forward problem consists of computing the observation $\mathbf{y}$ from the data sample $\mathbf{x}$ [17].

The inverse problem is the reverse problem. Given an observation $\mathbf{y}$, the aim is to infer the unknown data sample $\mathbf{x}$ from which it was produced. However, the forward operator typically destroys information and is therefore not invertible. Several data samples may consequently be consistent with the same observation. Therefore, bayesian inverse problem solving instead aims to draw samples from

the posterior distribution

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{x})p_{\text{data}}(\mathbf{x})}{p_Y(\mathbf{y})} \quad (2.2)$$

where $p(\mathbf{y} \mid \mathbf{x})$ is the likelihood of observing $\mathbf{y}$ given $\mathbf{x}$. Under the Gaussian measurement model in Equation 2.1, the likelihood is

$$p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(\mathbf{y} \mid \mathcal{A}(\mathbf{x}), \sigma_y^2 \mathbf{I}_M) \propto \exp\left(-\frac{\|\mathbf{y} - \mathcal{A}(\mathbf{x})\|_2^2}{2\sigma_y^2}\right) \quad (2.3)$$

Substituting the likelihood into Equation 2.2 gives

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{y}) \propto p_{\text{data}}(\mathbf{x}) \exp\left(-\frac{\|\mathbf{y} - \mathcal{A}(\mathbf{x})\|_2^2}{2\sigma_y^2}\right) \quad (2.4)$$

The posterior therefore assigns high density to samples that are both plausible under $p_{\text{data}}(\mathbf{x})$ and consistent with the observation $\mathbf{y}$.

2.1.3 Relation to Reward Alignment

Conditional generation can be viewed as a particular instance of the broader task of reward alignment. This connection allows us to discuss general approaches for reward alignment in the related-work chapter with the view that they can be applied to conditional generation tasks, such as inverse problem solving.

Reward alignment aims to generate samples from the target distribution with specified desirable properties, as quantified by a reward function $r(\mathbf{x})$. For example, a reward function may compute an aesthetic rating of an image. Reward alignment can be formalised as sampling from

$$p_{\text{reward}}(\mathbf{x}) \propto p_{\text{target}}(\mathbf{x}) \exp(r(\mathbf{x})) \quad (2.5)$$

This distribution reweights the target distribution towards samples with higher reward, and is known as the reward-tilted distribution [30–32].

To express conditional generation as reward alignment, we let each condition $\mathbf{y}$ define a corresponding reward function $r_{\mathbf{y}}(\mathbf{x})$. In particular, we choose this reward to be the log-likelihood of the condition given the sample $r_{\mathbf{y}}(\mathbf{x}) = \log p(\mathbf{y} \mid \mathbf{x})$. The corresponding reward-tilted distribution is

$$p_{\text{reward}}(\mathbf{x} \mid \mathbf{y}) \propto p_{\text{target}}(\mathbf{x})p(\mathbf{y} \mid \mathbf{x}) \propto p_{\text{target}}(\mathbf{x} \mid \mathbf{y}) \quad (2.6)$$

For the inverse problems defined above, $p(\mathbf{y} \mid \mathbf{x})$ is the likelihood in Equation 2.3. In this example, samples receive a high reward when they are consistent with the observation. Inverse problem solving is therefore a particular instance of

conditional generation, which can itself be formulated as reward alignment [24].

2.2 Flow-Based Generative Modelling

Flow-based generative models transform samples from a tractable base distribution $p_{\text{base}}(\mathbf{z})$ into samples from a target distribution $p_{\text{target}}(\mathbf{x})$. Normalising flows represent this transformation as a finite composition of invertible maps [27]. Flow models and flow maps represent it as a continuous transformation governed by an ODE. While flow models learn the vector field defining the ODE [4, 10], flow maps instead aim to directly learn the integral of the ODE [33].

These models play different roles in this thesis. A pretrained flow map acts as the generative model that maps noise to data, while a normalising flow is later used as the noise adapter. Flow models provide the continuous-time framework from which flow maps are defined.

2.2.1 Normalising Flows

We now give an overview of normalising flows because noise adapter in our method is chosen to be a normalising flow. We make this choice because normalising flows can represent complex distributions while supporting both sampling and exact density evaluation, enabling variational training. We first define normalising flows and explain how their densities are evaluated. We then introduce autoregressive flows which is the specific variation of normalising flows which we use in the thesis. Finally we explain how normalising flows can be learned using forward or reverse KL divergence objectives.

Definition and Change of Variables

Constructing Expressive and Tractable Flows

Autoregressive Normalising Flows

Transformer Autoregressive Flows

Training Normalising Flows

Bibliography

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [2] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG]. URL: <https://arxiv.org/abs/1907.05600>.
- [3] Tero Karras et al. *Elucidating the Design Space of Diffusion-Based Generative Models*. 2022. arXiv: 2206.00364 [cs.CV]. URL: <https://arxiv.org/abs/2206.00364>.
- [4] Yaron Lipman et al. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: <https://arxiv.org/abs/2210.02747>.
- [5] Xingchao Liu, Chengyue Gong, and Qiang Liu. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022. arXiv: 2209.03003 [cs.LG]. URL: <https://arxiv.org/abs/2209.03003>.
- [6] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: <https://arxiv.org/abs/2112.10752>.
- [7] Jonathan Ho et al. *Video Diffusion Models*. 2022. arXiv: 2204.03458 [cs.CV]. URL: <https://arxiv.org/abs/2204.03458>.
- [8] Haohe Liu et al. *AudioLDM: Text-to-Audio Generation with Latent Diffusion Models*. 2023. arXiv: 2301.12503 [cs.SD]. URL: <https://arxiv.org/abs/2301.12503>.
- [9] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. *Stochastic Interpolants: A Unifying Framework for Flows and Diffusions*. 2025. arXiv: 2303.08797 [cs.LG]. URL: <https://arxiv.org/abs/2303.08797>.
- [10] Peter Holderrieth and Ezra Erives. *Introduction to Flow Matching and Diffusion Models*. 2026. arXiv: 2506.02070. URL: <https://diffusion.csail.mit.edu/>.

- [11] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. *Flow map matching with stochastic interpolants: A mathematical framework for consistency models*. 2025. arXiv: 2406.07507 [cs.LG]. URL: <https://arxiv.org/abs/2406.07507>.
- [12] Nicholas M. Boffi, Michael S. Albergo, and Eric Vanden-Eijnden. *How to build a consistency model: Learning flow maps via self-distillation*. 2025. arXiv: 2505.18825 [cs.LG]. URL: <https://arxiv.org/abs/2505.18825>.
- [13] Yang Song et al. *Consistency Models*. 2023. arXiv: 2303.01469 [cs.LG]. URL: <https://arxiv.org/abs/2303.01469>.
- [14] Zhengyang Geng et al. *Mean Flows for One-step Generative Modeling*. 2025. arXiv: 2505.13447 [cs.LG]. URL: <https://arxiv.org/abs/2505.13447>.
- [15] Prafulla Dhariwal and Alex Nichol. *Diffusion Models Beat GANs on Image Synthesis*. 2021. arXiv: 2105.05233 [cs.LG]. URL: <https://arxiv.org/abs/2105.05233>.
- [16] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. 2022. arXiv: 2207.12598 [cs.LG]. URL: <https://arxiv.org/abs/2207.12598>.
- [17] Gregory Ongie et al. *Deep Learning Techniques for Inverse Problems in Imaging*. 2020. arXiv: 2005.06001 [eess.IV]. URL: <https://arxiv.org/abs/2005.06001>.
- [18] Chenlin Meng et al. *On Distillation of Guided Diffusion Models*. 2023. arXiv: 2210.03142 [cs.CV]. URL: <https://arxiv.org/abs/2210.03142>.
- [19] Huayu Chen et al. *Visual Generation Without Guidance*. 2025. arXiv: 2501.15420 [cs.CV]. URL: <https://arxiv.org/abs/2501.15420>.
- [20] Hyungjin Chung et al. *Diffusion Posterior Sampling for General Noisy Inverse Problems*. 2024. arXiv: 2209.14687 [stat.ML]. URL: <https://arxiv.org/abs/2209.14687>.
- [21] Jiaming Song et al. “Pseudoinverse-Guided Diffusion Models for Inverse Problems”. In: *International Conference on Learning Representations*. 2023. URL: https://openreview.net/forum?id=9_gsMA8MRKQ.
- [22] Bingliang Zhang et al. “Improving Diffusion Inverse Problem Solving with Decoupled Noise Annealing”. In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, June 2025, pp. 20895–20905. URL: <http://dx.doi.org/10.1109/CVPR52734.2025.01946>.
- [23] Jeongsol Kim, Bryan Sangwoo Kim, and Jong Chul Ye. *FlowDPS: Flow-Driven Posterior Sampling for Inverse Problems*. 2025. arXiv: 2503.08136 [cs.CV]. URL: <https://arxiv.org/abs/2503.08136>.

- [24] Abbas Mammadov et al. *Variational Flow Maps: Make Some Noise for One-Step Conditional Generation*. 2026. arXiv: 2603.07276 [cs.CV]. URL: <https://arxiv.org/abs/2603.07276>.
- [25] David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. “Variational Inference: A Review for Statisticians”. In: *Journal of the American Statistical Association* 112.518 (Apr. 2017), pp. 859–877. URL: <http://dx.doi.org/10.1080/01621459.2017.1285773>.
- [26] Danilo Jimenez Rezende and Shakir Mohamed. *Variational Inference with Normalizing Flows*. 2016. arXiv: 1505.05770 [stat.ML]. URL: <https://arxiv.org/abs/1505.05770>.
- [27] George Papamakarios et al. *Normalizing Flows for Probabilistic Modeling and Inference*. 2021. arXiv: 1912.02762 [stat.ML]. URL: <https://arxiv.org/abs/1912.02762>.
- [28] Frank Noé et al. *Boltzmann Generators – Sampling Equilibrium States of Many-Body Systems with Deep Learning*. 2019. arXiv: 1812.01729 [stat.ML]. URL: <https://arxiv.org/abs/1812.01729>.
- [29] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
- [30] Jerry Y. Huang et al. *How to Guide Your Flow: Few-Step Alignment via Flow Map Reward Guidance*. 2026. arXiv: 2604.27147 [cs.LG]. URL: <https://arxiv.org/abs/2604.27147>.
- [31] Peter Potapchik et al. *Meta Flow Maps enable scalable reward alignment*. 2026. arXiv: 2601.14430 [stat.ML]. URL: <https://arxiv.org/abs/2601.14430>.
- [32] Peter Holderrieth et al. *Diamond Maps: Efficient Reward Alignment via Stochastic Flow Maps*. 2026. arXiv: 2602.05993 [cs.LG]. URL: <https://arxiv.org/abs/2602.05993>.
- [33] Sander Dieleman. *Learning the integral of a diffusion model*. 2026. URL: <https://sander.ai/2026/05/06/flow-maps.html>.

Appendix A

Supplementary Material

A.1 Change of Variables

A.2 Score functions and score matching

A.3 Additional Experiments

A.3.1 GMM Experiments

Capacity Reward Hack in SR Inverse Problem